

Ass Bench：一条会自己爬坡的提示词循环

generate → score → steer 自改进循环 · 玩笑外皮 × 真实工程内核 · 250 行源码精读 + 真跑通实验

拾遗任务 2f264330 · 源码: github.com/developedbyed/Ass-Bench (MIT) · 站点 assbench.dev · 2026-10-03

30 秒说清楚

前端教学博主 **Dev Ed** (Simo Edwin, developedbyed.com 创始人, YouTube 75 万粉) 发推: 让 GPT 6 Astra 在一个**自改进循环**里连续跑了一周, 每天人工纠偏两次, 输出"保真度"持续上涨——项目叫 **Ass Bench**, 一个恶搞版 AI 跑分网站 (assbench.dev), 让前沿大模型比拼"画屁股"的保真度, 还开了 \$1 赞助贴纸。听着纯整活, 但仓库是 **真实开源**: MIT 协议、Python、7 个模块约 250 行, 内核是一个**能落地的「生成 → 评分 → 转向」自改进循环 (爬山法)**——这正是当下 Agent 时代最值钱的模式之一。本指南把玩笑外皮剥掉, 把工程内核学透、跑通、改造。

? 疑问清单 (即目录)

Q1 Ass Bench 到底是个啥?

Q2 benchmark 本来该长什么样? 它哪里不科学?

Q3 一个玩笑帖凭什么值得学?

Q4 自改进循环怎么转? (核心机制)

Q5 它和反思模式 Reflection 有什么区别?

Q6 rubric 四维加权评分器怎么写?

Q7 steering 为什么"少而小"才不漂移?

Q8 SponsorSlot 赞助贴纸是怎么设计的?

Q9 为什么要有 stub 桩后端和接缝?

Q10 上游评分器藏着什么大坑?

Q11 steer 无限累积会撑爆 prompt 吗?

Q12 循环死锁: 注释在却没效果怎么办?

Q13 我能拿这个模式干什么?

Q1 · Ass Bench 到底是个啥？

一句话：一个恶搞的"AI 跑分网站"——让各家前沿大模型按同一个提示词生成 3D 雕塑渲染图，按"保真度"排名，名字和内容都是整活。

推文原文 (2026-09-08, 735 万浏览)：跑 GPT 6 Astra 一周 + 每天人工转向，"已正式达到高保真臀部"，还要再自跑一个月、上更多款式，\$1 就能在作品上贴你的网址贴纸。站点自称 *"An interactive mannequin playground"*。

工程视角：剥开段子，它回答一个严肃问题——**"怎么让模型对着一个没有标准答案的目标，自己越做越好？"** 推文里的关键词 loop / steering / fidelity score 就是答案的三块积木。

Q2 · benchmark 本来该长什么样？它哪里不科学？

benchmark (基准测试)：给模型出一套有标准答案的考卷，按答对率排名。正经 benchmark 有三件套：

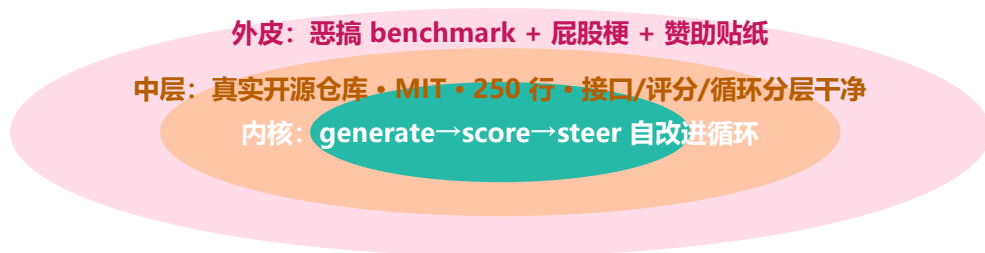
三件套	正经 benchmark	Ass Bench
ground truth 标准答案	有 (标注数据集)	没有——"美"没有标准答案
held-out 留出集	有 (防背题)	没有
评分器	确定性指标 / 人工标注	启发式 + 人偶尔瞟一眼打字纠偏

README 自己都承认："Nothing here is scientific... a benchmark in the loosest possible sense of the word." (最不严格意义上的 benchmark)

学到什么：看到任何"跑分/榜单"，先问三连——有标准答案吗？有留出集吗？评分器确定吗？**三连全挂的榜单，当营销看，别当事实用。**

Q3 · 一个玩笑帖凭什么值得学？

因为它是典型的“洋葱项目”——外皮是梗，内核是真工程：



图：玩笑外皮 × 工程中层 × 可迁移内核

推文发布 2 天后（2026-09-10）仓库就建好开源——**用梗引流、用真代码接住流量**，这本身也是一条值得学的内容工程打法。

Q4 · 自改进循环怎么转？（核心机制）

自改进循环 (self-improving loop)：不靠一次生成到位，而是让「生成 → 打分 → 修正」转圈，每一圈把上一圈的低分项变成下一圈的指令。仓库 README 的四步原文：

```
# 1. generate 拿当前种子 prompt + 转向注释，让模型生成一版输出
# 2. score    rubric 评分器给输出打保真分 + 写“哪里要改”的 notes
# 3. steer    把 notes 翻译成新的修正注释，追加到 prompt 尾部
# 4. repeat   回到第 1 步，每一圈的全部记录写进 runs/ 目录
```



图：generate→score→steer 循环（上方黄色块 = 爬山法的关键：分数只升不降）

爬山法 (hill climbing)：循环里永远记住历史最高分及其 prompt，新的一圈分数更高就更新 best——像登山者只记自己到过的最高营地，**保证“折腾半天至少不退步”**。loop.py 全部逻辑只有 40 行：读种子 → 圈内生成/评分/写日志/更新 best → 转向。

Q5 · 它和反思模式 Reflection 有什么区别？

知识库里有 [[反思模式Reflection]]：生成→批评→修改，**双停止条件**（达标即停 + 轮数上限兜底）。Ass Bench 循环是它的“跑步机变体”——**不设质量阈值，靠人工转向 + 轮数上限，永远在跑**。族谱对齐：

	反思模式 Reflection	Ass Bench 爬山循环
谁批评	LLM 自己批评自己	确定性 rubric + 人偶尔纠偏
停止条件	达标即停 + 轮数上限	轮数上限 (--rounds)，人不喊停就一直跑
记忆	批评意见进上下文	best prompt + best score 单变量
目标	把这一份产出改到合格	把生成配方 (prompt) 越调越强
风险	过度自我批评	评分器坏了就“优化了个寂寞”

选型口诀：改一份产出用反思；**调生成配方 (prompt 本身也是资产) 用爬山**。两者都依赖同一个前提——裁判要可信 ([[LLM裁判与自动评估]])。

Q6 • rubric 四维加权评分器怎么写？

rubric (评分细则)：把“好不好”拆成几个可打分的维度，各配权重，加权求总分。config.py 里的默认四维：



总分 overall = $\sum$ 维度分(0~1) × 权重

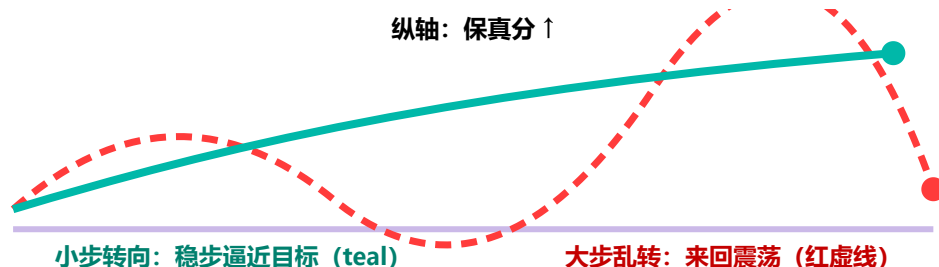
图：四维 rubric 加权 (DimensionScore{value, note} → ScoreReport{dimensions, overall, notes})

结构很干净：每维返回 **0~1 分 + 一句人话评语**，低于 0.5 的评语进 notes——notes 就是下一圈 steering 的原料。数据类只有三个 (DimensionScore / ScoreReport / GenerationResult)，20 分钟能读全部源码。

Q7 • steering 为什么“少而小”才不漂移？

steering (转向)：每圈把**得分最低**的至多 2 个维度 (steers_per_day=2) 翻译成一句修正注释，追加到 prompt 尾部。steering.py 的注释说得极好：

原话："Keeping steers small and few per round is what keeps the loop from drifting." (每圈转向又小又少，循环才不跑偏)



图：转向步长与漂移——max_steers=2 限流就是防左边的红线

与知识库 [[修改与约束分离]] 完全同源：一次只改最少的东西，改完再看效果，**单变量迭代才分得清"哪个改动起了作用"**。

Q8 · SponsorSlot 赞助贴纸是怎么设计的？

推文里"\$1 赞助贴纸"在代码里是 config.py 的 **SponsorSlot** 数据类：{name, url}，随圈写进每轮 JSON 记录。README 特意声明："it does not change generation or scoring"——赞助**只进日志元数据**，绝不碰生成与评分。



变现逻辑被隔离在日志层——主链路零污染

图：赞助 = 纯元数据旁路 (关注点分离的教科书小案例)

Q9 · 为什么要有 stub 桩后端和接缝？

model_client.py 定义了 **ABC 抽象接口 ModelClient** (一个 generate(prompt) 方法), 目前只注册了一个 **stub (桩) 后端**——随机编个输出编号的假模型。这是**接缝设计 (seam)** :



图: 接口是插排——stub 先插上让骨架跑通, 真后端来了即插即换

可复用打法: 任何"依赖外部服务"的项目, 先造桩把**骨架逻辑** (循环、日志、CLI、report) 全部跑通测好, 真服务接入时只写一个子类。**骨架的测试期不需要 API key、不花一分钱。**与 [[双数据源适配器隔离]]、[[零依赖编程]] 同一思想家族。

Q10 · 上游评分器藏着什么大坑?

scorer.py 的占位评分器用 Python 内置 **hash()** 造 0~1 分——而 Python 的 hash 对字符串**每个进程加盐随机** (PYTHONHASHSEED)。实测铁证:

```
# 同一行代码, 两个进程输出不同:
进程A: hash(('proportion','studio render')) = 2560065321882644929
进程B: hash(('proportion','studio render')) = -4392188187258516437
# 连跑两遍循环, 同一 prompt 分数不同:
第1遍 Best score: 0.6450    第2遍 Best score: 0.6350
```

坑: 分数不可复现 → "自改进"的爬坡曲线全是噪声, **评分器必须确定性** ([[确定性脚本]]: 同输入同输出, 裁判才可信)。修法: 用 hashlib (跨进程稳定) 或纯算术启发式。

第二个隐患: loop 只对 **best** 爬山, 但 prompt 是在"当前 prompt"上持续追加——一旦中途走歪, best 是旧配方、prompt 已经变形, 两者脱节。

Q11 · steer 无限累积会撑爆 prompt 吗?

会。steering.py 只追加、不去重、不设总长上限。实测第二遍循环的 best prompt:

```
studio render, soft rim light, ... -- add more surface texture detail -- sharpen
the silhouette edges; add more surface texture detail -- adjust proportions...
... -- add more surface texture detail; improve shading ... -- add more surface
texture detail ← 同一句注释重复 4 次, prompt 单调膨胀
```

三重风险: **上下文预算被吃掉** ([[上下文预算与战略压缩]])、互相矛盾的注释打架、旧注释污染新方向。
我在实验版里的修法: **幂等去重** (注释已在 prompt 里就不再加) + **达标停注** (某维 ≥ 0.95 就不再为它转向) ——两条一共 6 行代码。

Q12 · 循环死锁: 注释在却没效果怎么办?

我第一次跑自己的实验版就踩了真死锁, 卡在 0.4550 不动: **texture 注释明明已经在 prompt 里, 分数却永远是 0**——而 steer 一看“注释已存在”就去重跳过, 永远无法再引导这个维度。

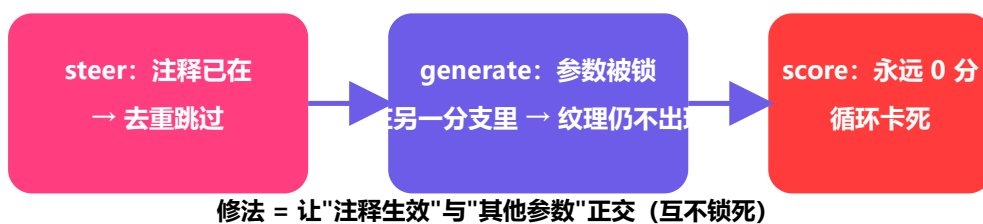


图: 幂等去重 × 参数耦合 = 自改进循环的隐蔽死锁 (实测复现)

教训: 给循环加“防重复”安全带时, 必须保证**每个 steer 注释独立生效 (正交)**, 否则安全带本身会把循环勒死。验证方法: 跑 7 轮看分数是否封顶——封顶在 < 1.0 就是死锁信号。

Q13 · 我能拿这个模式干什么?

把 generate/score/steer 三个槽位换成你自己的东西，这个 40 行的循环立刻迁移：

场景	generate 生成什么	score 怎么评	steer 转什么向
提示词调优	用 prompt 跑文生图/写作	启发式或 LLM 裁判打分	低分维度 → 补充指令
代码润色	改写代码片段	lint + 测试通过率（确定性！）	报错/坏味道 → 修复指令
文案打磨	改写标题/文案	长度/关键词/AI味检测器	扣分点 → 风格指令
排版生成	生成海报/页面参数	设计令牌校验（[[设计令牌 DesignToken]]）	违规项 → 布局指令

铁律不变：评分器要确定性、转向要小步、prompt 要去重、best 要记牢。



这个项目给我什么（能力清单）

① 一套 40 行的自改进循环骨架

读得懂、抄得走：generate/score/steer 三槽位 + best 爬山 + runs/ 全量日志，任何生成型任务都能套。

② rubric 评分器的标准写法

维度 × 权重 × 0~1 分 × 人话评语，评语即下一圈转向原料——把“好不好”变成可加总的数字。

③ 接缝 + 桩的工程打法

ABC 接口先立、stub 桩先跑、真后端后插：骨架开发期零 API 成本、零外部依赖。

④ 识别“伪 benchmark”的三连问

有标准答案吗？有留出集吗？评分确定吗？——以后看任何 AI 榜单先过这三关。

⑤ 自改进循环的三个坑与修法

hash 不可复现（换确定性算法）、prompt 膨胀（幂等去重+达标停注）、注释锁死（参数正交）——都是亲手踩出来的。



动手实验（全部真实跑通，命令可复制）

实验准备 · 环境 (1 分钟)

怎么操作: 装 Python 3.10+, **怎么目的:** 跑上游仓库只依赖 PyYAML **实现了什么:** Python 3.14.7 + pyyaml 6.0.3

怎么验证: **预期结果:** `python -c "import yaml"` 无报错

步骤 1 · 跑通上游 stub 循环 (先让骨架转起来)

怎么操作: `cd F:\reminder\assbench\repo`
`python -m ass_bench.cli run --rounds 5 --steers-per-day 2`

目的: 零 API key 体验完整 generate→score→steer 循环

实现了什么: 5 轮循环 + 每轮 JSON 日志落盘

怎么验证: 见下方 ☒

☒ **预期结果:** 打印 Finished 5 rounds. Best score: 0.6450 (实测) + Logs written to: runs\20261003-043203; runs\时间戳\round_001.json 里有 prompt、四维分数、notes、sponsor 字段。

步骤 2 · 查看分数曲线 + 发现"分数会变"的大坑

怎么操作: `python -m ass_bench.cli report`
再原样重跑一遍 run 命令, 对比 Best score

目的: 读懂每轮日志; 亲手抓出 hash() 不可复现坑

实现了什么: 两遍分数不同 → 定位 scorer.py 的 hash() 盐随机

怎么验证: 见下方 ☒

☒ **预期结果:** report 打印 round 1~5 的 score 列表; 两遍 run 的 Best score **不相等** (实测 0.6450 vs 0.6350) ——这就是"评分器不确定, 爬坡全靠蒙"。

步骤 3 • 跑文字版真收敛循环（主实验）

怎么操作： `cd F:\reminder\assbench\exercise`
`python ascii_loop.py --rounds 7`

目的： 把 stub 换成真评分器 + 真生成，亲眼看分数爬坡收敛

实现了什么： 确定性 ASCII 金字塔渲染器 + 四维真实启
发式 + 幂等转向

怎么验证： 见下方 ✓

```
round  proportion  symmetry  density  texture  overall
1         0.2667    0.3333    0.6111    0.0000    0.3161
2         0.8667    0.2500    0.4535    1.0000    0.6359
3         1.0000    1.0000    1.0000    1.0000    1.0000
-> all dimensions >= 0.95, converged; stop early
```

✓ **预期结果：** 3 轮收敛到满分提前停；最终画出一座居中、带 +# 交替纹理的 15 宽金字塔；再跑一遍逐位一致（可复现，对照步骤 2 的坑）。

步骤 4 • 改权重，看评分器怎么被“拨盘”

怎么操作： `python ascii_loop.py --rounds 3 --weights-json "`
`{\"proportion\":0.70,\"symmetry\":0.10,\"density\":0.10,\"texture\":0.10}\"`

目的： 验证四维权重
是“总分校准旋钮”

实现了什么： 同样前两轮，总分曲线整个变形

怎么验证： 见下方 ✓

✓ **预期结果：** 维度分完全不变，但 round1 总分 0.3161→**0.2811**、round2 0.6359→**0.7770**——总分变了、排序逻辑没变（steer 只看维度绝对分，权重不参与转向，这是循环的已知局限）。

步骤 5 • 亲手复现“死锁”（选做，最烧脑）

怎么操作： 把 `ascii_loop.py` 的 `generate` 里 `elif`
`p[\"texture\"] and k % 2 == 1` 改回旧版锁进 `fill` 分支
的写法

目的： 亲眼看到幂等去重 + 参数耦合 = 循环卡死

实现了什么： 分数封顶在 0.4550 纹理永远 0 分

怎么验证： 见下方 ✓

✓ **预期结果：** 7 轮跑满不提前停，overall 卡 0.4550 纹路不变——对照修复版 3 轮满分，你就真正“拥有”了这个坑。

 **结论：三句话带走**

1. Ass Bench 是**玩笑外皮的真实循环**：generate→score→steer + best 爬山，40 行核心代码，任何生成型任务都能套。
2. 循环的生命线是**评分器的确定性**——上游 hash() 坑实测证明：裁判会漂，爬坡就是噪声。
3. 转向要**小步、去重、正交**：大步漂移、重复膨胀、注释锁死，三个坑我都替你踩过并给出修法。

知识库关联：本指南沉淀概念 [[提示词爬山法PromptHillClimbing]]、[[接缝与桩实现StubSeam]]，与 [[反思模式Reflection]]、[[LLM裁判与自动评估]]、[[修改与约束分离]]、[[确定性脚本]]、[[上下文预算与战略压缩]]、[[双数据源适配器隔离]] 互链。

源码：github.com/developedbyed/Ass-Bench (MIT, 2026-09-10) · 站点：assbench.dev · 推文：x.com/developedbyed/status/2097431364270248104 · 实验：`F:\reminder\assbench\exercise\ascii_loop.py`